



Universidad Don Bosco

FACULTAD DE INGENIERÍA

ESCUELA DE COMPUTACIÓN

Diseño y Programación de Software Multiplataforma

Guía 8: Introducción al desarrollo de aplicaciones Web con ASP.NET MVC



I. Objetivos

Al finalizar la guía, el estudiante será capaz de:

- Comprender los fundamentos de ASP.NET y su aplicación en el desarrollo web.
- Entender la arquitectura de aplicaciones ASP.NET y el patrón Modelo-Vista-Controlador (MVC).
- Desarrollar aplicaciones web básicas utilizando ASP.NET MVC.
- Aplicar conceptos aprendidos.
- Fomentar la investigación y el aprendizaje autónomo mediante ejercicios complementarios.

II. Introducción Teórica

a. ¿Qué es ASP.NET?

ASP.NET es un marco de trabajo (framework) desarrollado por Microsoft para construir aplicaciones web dinámicas, robustas y escalables. Permite a los desarrolladores crear aplicaciones utilizando lenguajes como C# y herramientas como Visual Studio. ASP.NET MVC es una variante que implementa el patrón MVC para separar la lógica de negocio, la interfaz de usuario y la interacción del usuario.

Las más importantes características que se tiene son:

- **Código abierto y multiplataforma:**

ASP.NET es de código abierto y funciona en diferentes sistemas operativos, como Windows, macOS y Linux, además de en contenedores como Docker.

- **Basado en .NET:**

ASP.NET se construye sobre la plataforma .NET de Microsoft, que proporciona un conjunto de herramientas, bibliotecas y lenguajes para el desarrollo de aplicaciones.

- **Varias formas de desarrollo:**

ASP.NET ofrece diversas formas de desarrollo, como ASP.NET Core, que es una reimplementación modular y multiplataforma, y Razor Pages, que facilita el desarrollo de aplicaciones web centradas en páginas.

- **Amplia gama de aplicaciones:**

ASP.NET se utiliza para crear una variedad de aplicaciones web, como sitios web dinámicos, servicios web (APIs), aplicaciones en tiempo real, aplicaciones para Internet de las cosas (IoT) y backends móviles.

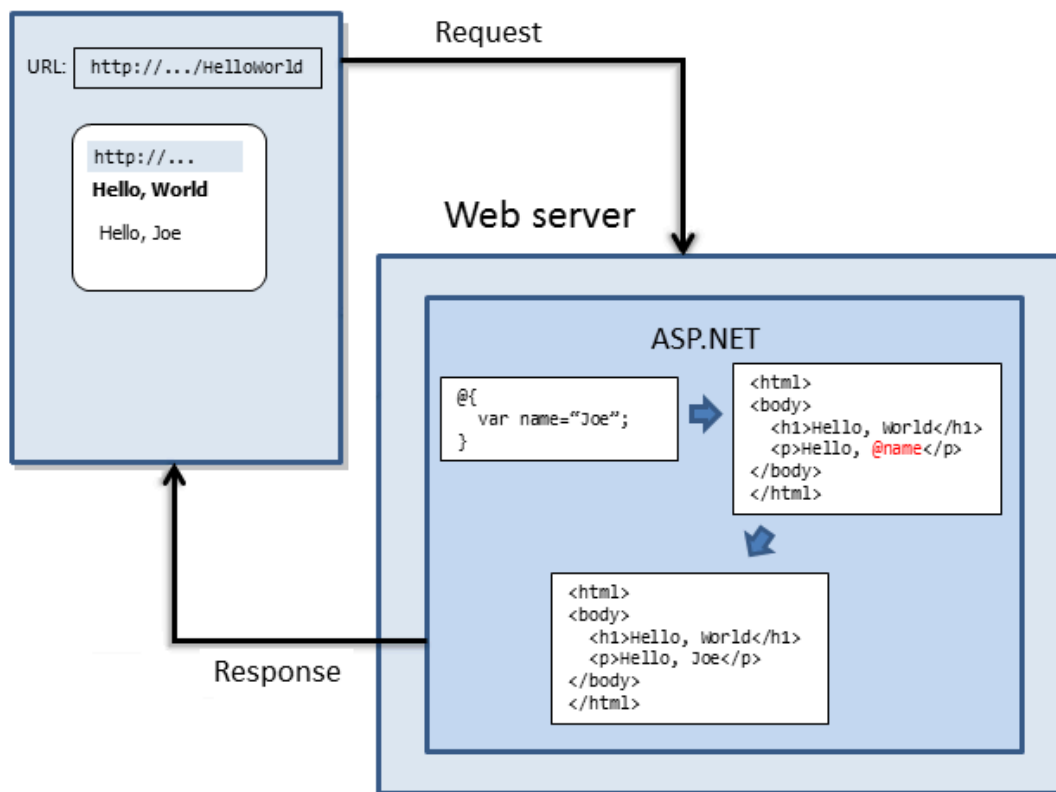
- **Desarrollo web moderno:**

ASP.NET Core, en particular, está diseñado para el desarrollo web moderno, con enfoque en el rendimiento, la capacidad de prueba y la integración con tecnologías del lado del cliente, como JavaScript.

- **Integración con otras tecnologías:**

ASP.NET se integra con otras tecnologías como SignalR para el desarrollo en tiempo real, Web API para la creación de APIs RESTful y Blazor para usar C# en el navegador.

Browser (Client)



b. Arquitectura de Aplicaciones ASP.NET

ASP.NET se basa en una arquitectura cliente-servidor, donde el servidor procesa las solicitudes HTTP y genera respuestas dinámicas. Sus componentes principales incluyen:

- **Common Language Runtime (CLR):** Ejecuta el código y gestiona recursos.
- **Biblioteca de clases de .NET:** Proporciona clases reutilizables para tareas comunes.
- **Middleware:** Gestiona las solicitudes y respuestas HTTP.
- **Razor:** Motor de vistas para generar HTML dinámico.

También la arquitectura de aplicaciones ASP.NET puede referirse a varios patrones y estructuras, pero comúnmente se basa en la arquitectura en capas o en la arquitectura limpia, especialmente en ASP.NET Core. Estas arquitecturas se centran en la separación de preocupaciones y la reducción de dependencias para crear aplicaciones robustas y mantenibles.

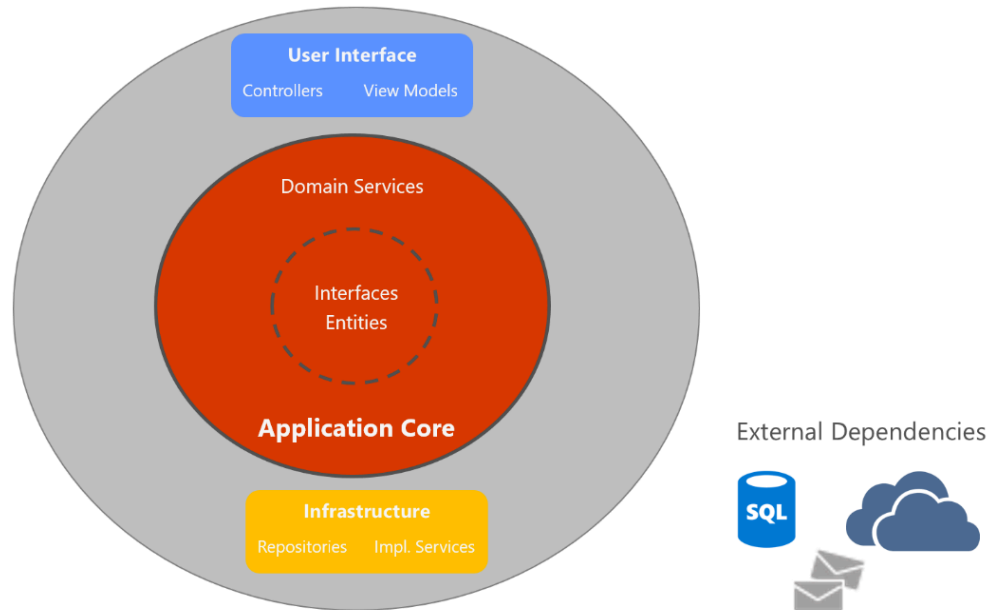
Arquitectura en Capas:

- **Capa de Presentación:** Maneja la interacción con el usuario (interfaz de usuario, controladores, vistas).
- **Capa de Aplicación:** Contiene la lógica de negocio específica de la aplicación.
- **Capa de Dominio:** Define las entidades y reglas de negocio centrales.
- **Capa de Infraestructura:** Interactúa con sistemas externos (bases de datos, servicios web, etc.).

Arquitectura Limpia (Cebolla):

- Similar a la arquitectura en capas, pero con un enfoque más estricto en la separación de dependencias.
- El núcleo de la aplicación (capa de dominio y aplicación) es independiente de la infraestructura.
- Las capas externas (infraestructura, presentación) dependen de las capas internas, pero no viceversa.

Clean Architecture Layers (Onion view)



Otros aspectos importantes de la arquitectura ASP.NET:

- **MVC (Modelo-Vista-Controlador):**

Un patrón de diseño común en aplicaciones web ASP.NET, que divide la aplicación en tres componentes principales: modelos (datos), vistas (presentación) y controladores (lógica).

- **Microservicios:**

ASP.NET Core es adecuado para construir aplicaciones basadas en microservicios, donde las aplicaciones se dividen en pequeños servicios independientes.

- **Contenedores:**

ASP.NET Core es compatible con la contenerización (Docker) y la orquestación (Kubernetes), lo que facilita la implementación y gestión de aplicaciones en la nube.

- **Enfoque en la nube:**

ASP.NET Core está diseñado para la nube, con características como la configuración basada en el entorno y la escalabilidad.

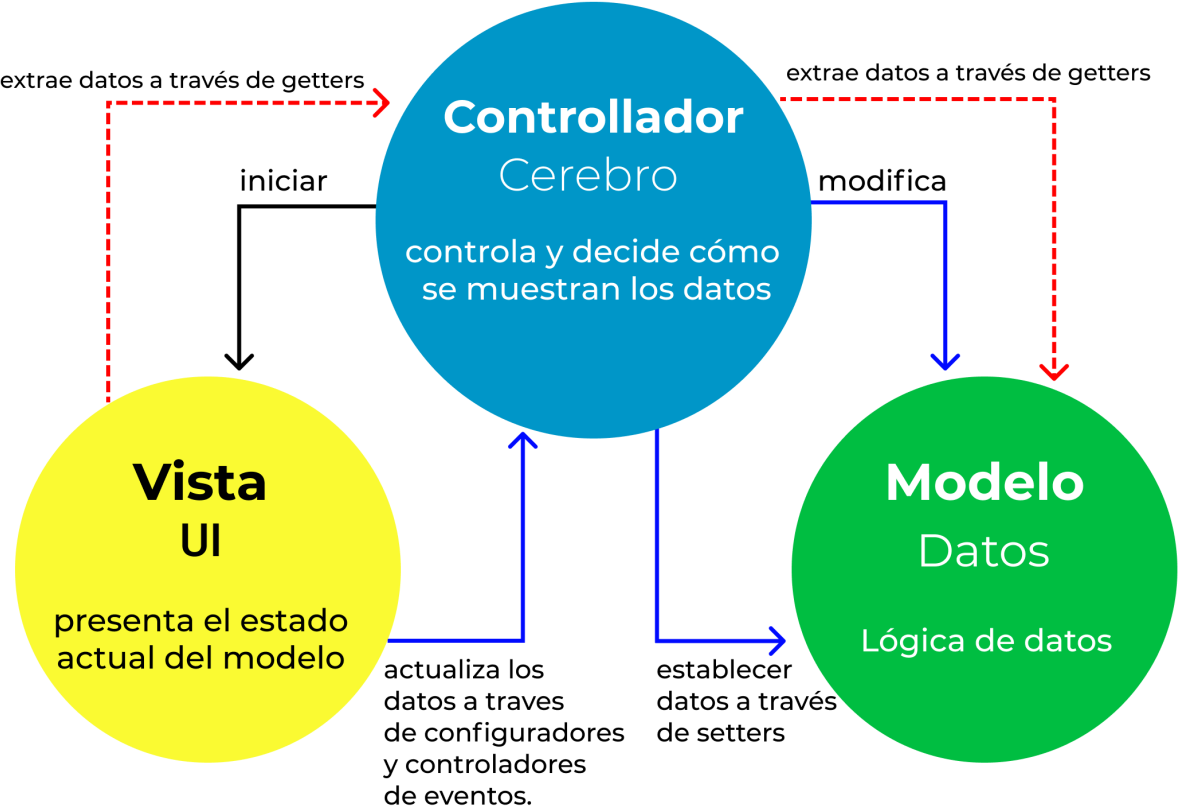
En resumen, la arquitectura de aplicaciones ASP.NET puede variar, pero generalmente se basa en la separación de preocupaciones (capas) y la reducción de dependencias, con patrones como MVC y enfoques como microservicios y contenedores para construir aplicaciones modernas y escalables.

c. Modelo Vista Controlador (MVC)

El patrón MVC divide la aplicación en tres componentes:

- **Modelo:** Representa los datos y la lógica de negocio.
- **Vista:** Define la interfaz de usuario.
- **Controlador:** Maneja las solicitudes del usuario, interactúa con el modelo y selecciona la vista adecuada.

Esta separación mejora la modularidad, el mantenimiento y la capacidad de realizar pruebas unitarias.



III. Requerimientos

Nº	Cantidad	Descripción
1	1	Visual Studio 2022 Versión Community o superior
2	1	Guía 4
3	1	Memoria USB o almacenamiento en la nube.

Ejemplo 1 : Crear una aplicación MVC básica

Este ejercicio es una introducción al desarrollo de sitios web con **ASP.NET Core MVC**. Se desarrollará una simulación de un cajero automático en línea con autenticación básica y operaciones simples sobre el saldo de una cuenta. A continuación, se diseña una simulación de un cajero ATM on-line, donde el usuario realiza transacciones con el saldo de su cuenta posterior a una autenticación básica que se realiza en el sitio.

1. Proceda a ejecutar Visual Studio en su computadora.
2. En la ventana de selección para comenzar a utilizar Visual Studio, de clic en *Crear un Proyecto*.
3. Luego, tal como se muestra en la imagen 7.1, filtre las plantillas de proyectos, eligiendo en los cuadros de combos de *Lenguaje*, *Plataforma* y *Tipo de proyecto*, a las opciones **C#**, **Windows** y **Web**, respectivamente.

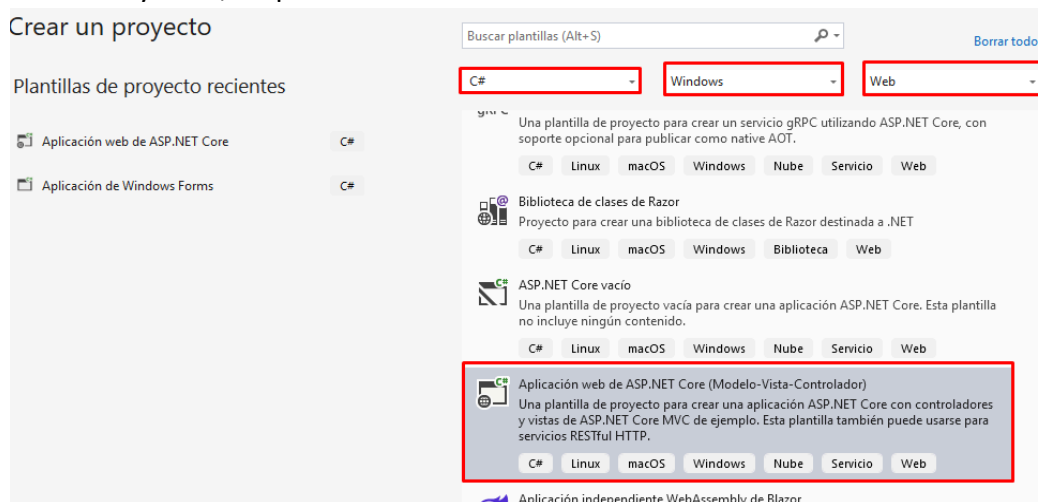
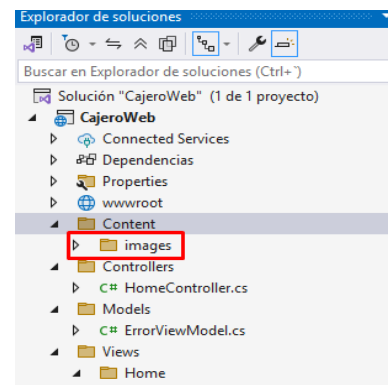


Imagen 7.1: Opciones para filtrar plantillas de proyectos de sitios web con ASP.

4. Luego, de las plantillas devueltas, seleccione *Aplicación web ASP.NET Core* y de clic en botón *Siguiente*.
5. En el asistente para configurar el Proyecto, defina el nombre de proyecto **CajeroWeb**, así como elegir al **FrameWork 8 o superior** (ver Imagen 7.2). De clic en botón *Crear*.

Imagen 7.2: Tipo de plantilla de la aplicación web ASP a crear.

6. Proceda a crear una nueva carpeta en la raíz del sitio. Para ello, en la ventana del *Explorador de soluciones*, de clic secundario sobre el nombre del proyecto (CajeroWeb); del menú contextual, seleccione la opción *Agregar* y luego *Nueva carpeta*. A la nueva carpeta a crear en la raíz, asígnele nombre **Content**.
7. Una vez creada la carpeta **Content**, agregue en su interior a las carpetas **imágenes**, proporcionados en los recursos de esta guía.
8. Confirmar que se incorporó correctamente las imágenes en el proyecto
9. Agregue un nuevo modelo a su proyecto web. Para ello, desde la ventana del *Explorador de soluciones*, ubique y haga clic secundario sobre la carpeta **Models**. Del menú contextual, seleccione la opción *Agregar* y luego *Clase*. Defina al nombre de clase **transaccion** y de clic en botón *Agregar*.
10. Dentro del cuerpo de esta nueva clase **transaccion**, proceda a digitar el siguiente código:



```
namespace CajeroWeb.Models
{
    namespace CajeroWeb.Models
    {
        // Clase que se utiliza para gestionar las transacciones a realizar por el cajero ATM
    }
}
```

```

public class Transaccion
{
    // Campos privados
    private string sNumeroTarjeta;
    private double fSaldo;

    // Constructor que acepta solo el número de tarjeta
    public Transaccion(string tarjeta)
    {
        sNumeroTarjeta = tarjeta;
        fSaldo = 500;
    }
    // Constructor que acepta número de tarjeta y saldo
    public Transaccion(string tarjeta, double saldo)
    {
        sNumeroTarjeta = tarjeta;
        fSaldo = saldo;
    }
    // Métodos de acceso (getters y setters)
    public double GetSaldo()
    {
        return fSaldo;
    }
    public void SetSaldo(double saldo)
    {
        fSaldo = saldo;
    }
    public string GetNumTarjeta()
    {
        return sNumeroTarjeta;
    }

    public void SetNumTarjeta(string tarjeta)
    {
        sNumeroTarjeta = tarjeta;
    }
    // Métodos para realizar transferencias
    public void TransferenciaNacional(double monto)

```

```

        {
            fSaldo -= monto;
        }
        public void TransferenciaInternacional(double monto)
        {
            fSaldo -= monto;
        }
    }
}
}

```

11. Proceda a crear un nuevo controlador para la pagina principal. Para ello, ubique en Explorador de soluciones a la carpeta **Controller** y de clic secundario sobre la misma. Del menú contextual, seleccione la opción *Agregar* y luego *Controlador*.
12. Cree un controlador vacío, seleccionando la opción **Controlador de MVC: en blanco** y luego de clic en boton *Agregar*.
13. Dentro de la nueva clase **HomeController**, realice los siguientes cambios:
Primero, con la directiva **using**, incluya a las clases almacenadas en la subcarpeta **Models** del namespace (CajeroWeb) de su proyecto, digitando a la siguiente sentencia antes de la declaración del namespace del proyecto:

using CajeroWeb.Models;

14. Luego, dentro del cuerpo de la clase, redacte la siguiente definición de sus miembros:

```

using CajeroWeb.Models.CajeroWeb.Models;
using Microsoft.AspNetCore.Mvc;
using System.Diagnostics;
namespace CajeroWeb.Controllers
{
    public class HomeController : Controller
    {
        [HttpGet]
        public IActionResult Index()
        {
            return View();
        }
    }
}

```

```
[HttpPost]
public IActionResult Index(string sNumeroTarjeta, string sNumeroPIN)
{
    double sSaldo;
    string numero;
    Transaccion nuevaTransaccion = new Transaccion(sNumeroTarjeta);
    if (sNumeroTarjeta == "123456" && sNumeroPIN == "654321")
    {
        numero = nuevaTransaccion.GetNumTarjeta();
        sSaldo = nuevaTransaccion.GetSaldo();
        return Redirect("~/Menu/Menu?sNumeroTarjeta=" + sNumeroTarjeta + "&sSaldo=" +
sSaldo);    }
```

```

else
{
    // Si el numero de pin o de tarjeta no corresponde, entonces se
    // procede a mostrar un mensaje de error
    ViewBag.Error = "Error en número de tarjeta o número de PIN. Verifique";
    return View();
}
}

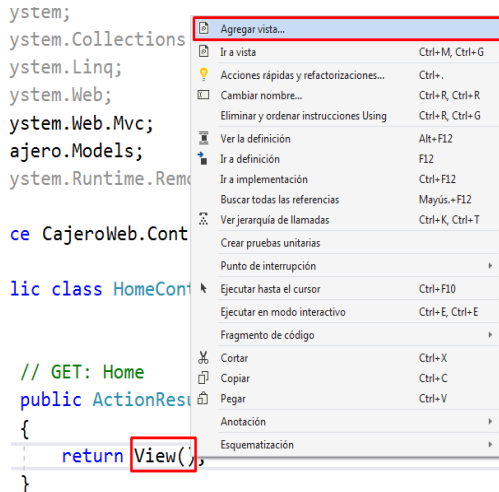
public IActionResult Privacy()
{
    return View();
}

[ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
public IActionResult Error()
{
    return View(new ErrorViewModel { RequestId = Activity.Current?.Id ??
HttpContext.TraceIdentifier });
}
}
}

```

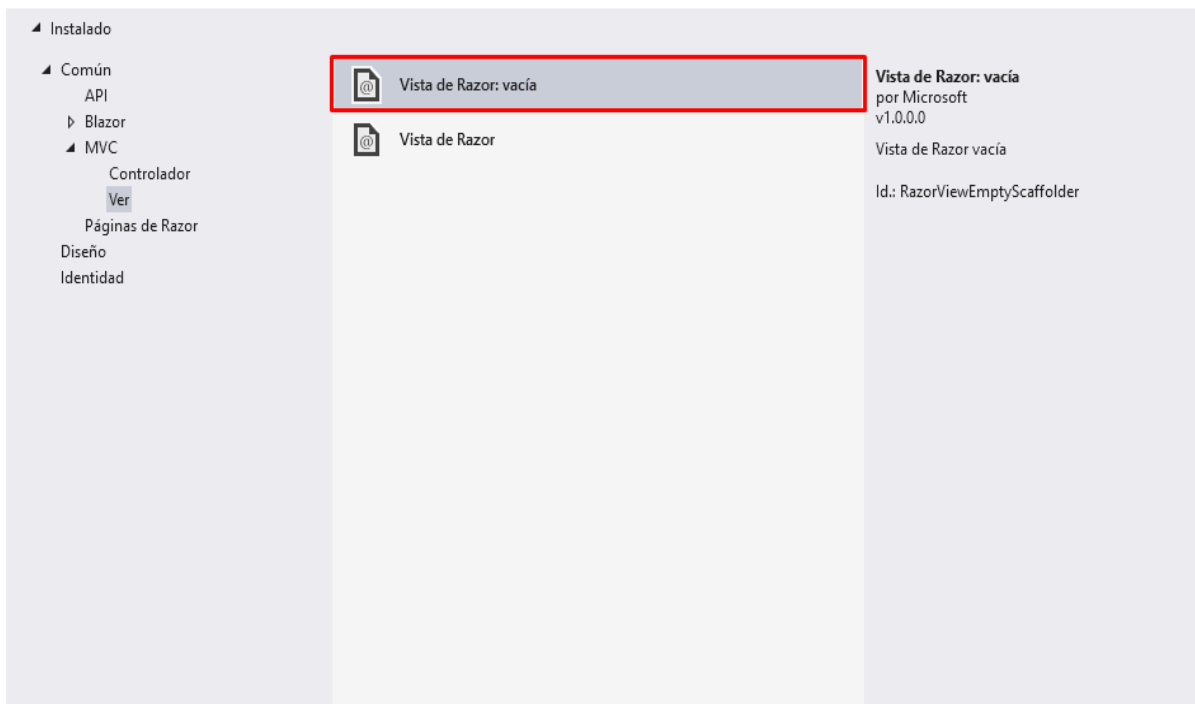
15. Ahora procederá a crear una vista para la página principal del website.

Dentro de la clase **HomeController**, localice y de clic secundario sobre la llamada a su método **View**. Del menú contextual, seleccione la opción *Agregar vista..* tal como se muestra en la imagen a continuación:

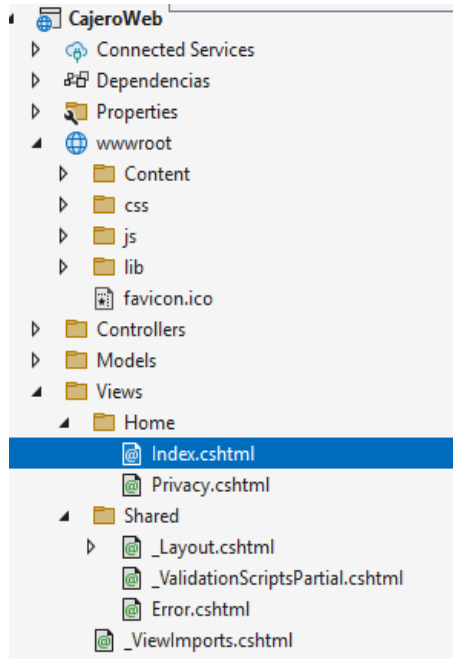


16. En la ventana *Agregar vista*, deje el nombre por defecto (**Index**) para la Vista y el tipo de Plantilla predeterminado **Vacia (sin modelo)**, para hacer clic en *Agregar*.

Agregar nuevo elemento con scaffolding



17. Confirme que dentro de la ruta de carpetas **View/Home**, se agrega automáticamente a un archivo de C# con extensión **cshtml** llamado **Index.cshtml** y se abre su ventana para edición de código.



18. Dentro de la ventana de edición del archivo **index.cshtml**, reemplace el código predeterminado por el siguiente código:

```
@{
    ViewBag.Title = "Index";
}

<div class="container mt-5">
    <div class="row">
        <!-- Sección izquierda: formulario -->
        <article class="col-md-7">
            <h2 class="mb-4">Bienvenido</h2>
            <p>
                Innovando las transacciones en línea, ponemos a su disposición el cajero ATM online
                donde podrá realizar sus pagos y transferencias de forma rápida y fácil,
                siempre con la calidad y la confianza que nos caracterizan. Realice sus transacciones
                desde la comodidad de su hogar las 24 horas del día, los 7 días de la semana.
            </p>

            @using (Html.BeginForm("Index", "Home", FormMethod.Post))
```

```

    {
        <div class="mb-3">
            @Html.Label("sNumeroTarjeta", "Tarjeta #", new { @class = "form-label" })
            @Html.TextBox("sNumeroTarjeta", null, new { @class = "form-control", placeholder =
"Ingrese su número de tarjeta" })
        </div>

        <div class="mb-3">
            @Html.Label("sNumeroPIN", "PIN", new { @class = "form-label" })
            @Html.Password("sNumeroPIN", null, new { @class = "form-control", placeholder =
"Ingrese su PIN" })
        </div>

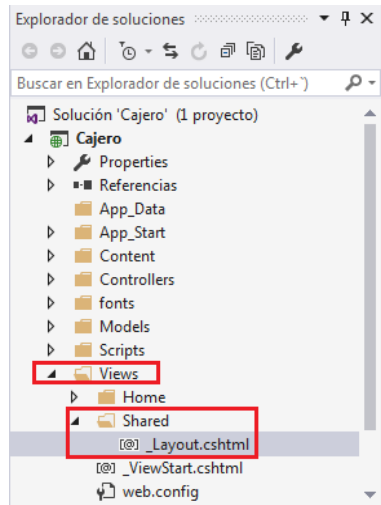
        @if (ViewBag.Error != null)
        {
            <div class="alert alert-danger">
                @ViewBag.Error
            </div>
        }

        <div class="d-grid">
            <button type="submit" class="btn btn-primary">Ingresar</button>
        </div>
    }
</article>

<!-- Sección derecha: imagen -->
<article class="col-md-5 text-center">
    
</article>
</div>
</div>

```

19. Ahora, proceda a modificar la plantilla principal de su proyecto. Para ello, abra la carpeta **View -> Shared** y ubique el archivo **Layout.cshtml**.



20. Dentro de este archivo **Layout.cshtml**, reemplace el código predeterminado por el siguiente código:

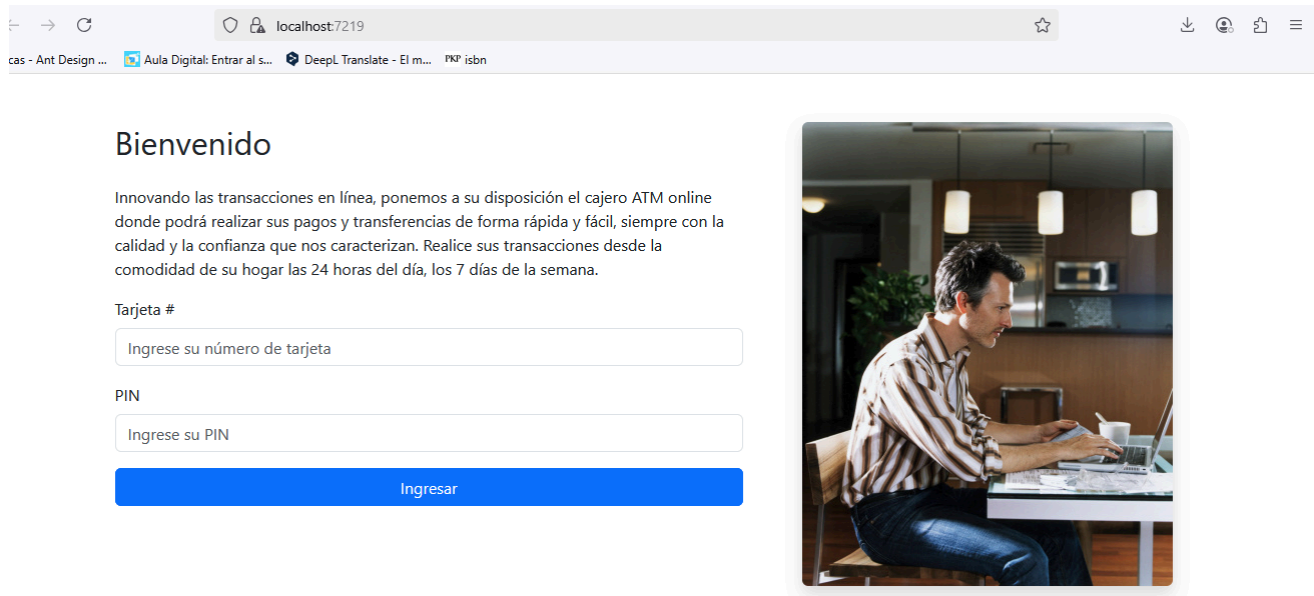
```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewBag.Title - CajeroWeb</title>

  <!-- Bootstrap 5 desde CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" />

  <!-- Tu CSS personalizado si tienes -->
  <link href="~/Content/site.css" rel="stylesheet" />
</head>
<body>
  <div class="container">
    @RenderBody()
  </div>

  <!-- Bootstrap JS (opcional si usas modales, dropdowns, etc.) -->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

21. Ejecute la aplicación web. Luego de unos segundos, se cargara la siguiente ventana de contenido en una ventana del navegador predeterminado:



22. Cree un nuevo controlador y asígnele el nombre **MenuController**. Luego, dentro del cuerpo de la clase **MenuController** correspondiente de este controlador, reemplace el método Index predeterminado por el siguiente método denominado Menu:

```
using Microsoft.AspNetCore.Mvc;

namespace CajeroWeb.Controllers
{
    public class MenuController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        // GET: Menu
        public ActionResult Menu(string sNumeroTarjeta, double sSaldo)
        {
            ViewBag.sNumeroTarjeta = sNumeroTarjeta;
            ViewBag.sSaldo = sSaldo;
            return View();
        }
    }
}
```

22. Crear una nueva Vista llamada **Menu**, basada en el Controlador **MenuController** creado en paso anterior.

Luego, digite el siguiente código en el correspondiente archivo **Menu.cshtml** de la nueva Vista.

```
@{
    ViewBag.Title = "Menu";
}

<div class="container mt-5">
    <div class="row">
        <!-- Sección principal del mensaje de bienvenida -->
        <div class="col-md-8">
            <div class="card shadow mb-4">
                <div class="card-body">
                    <h2 class="card-title mb-3">Bienvenido</h2>
                    <p class="card-text">
                        Innovando las transacciones en línea, ponemos a su disposición el cajero ATM online,
                        donde podrá realizar sus pagos y transferencias en línea de una forma rápida y fácil,
                        siempre con la calidad y la confianza que nos caracterizan.

                        Realice sus transacciones desde la comodidad de su hogar las 24 horas del día,
                        los 7 días de la semana.
                    </p>
                </div>
            </div>
        </div>

        <!-- Sección de navegación -->
        <div class="col-md-4">
            <div class="card shadow">
                <div class="card-header bg-primary text-white">
                    <strong>Opciones del menú</strong>
                </div>
            </div>
        </div>
    </div>
</div>
```

```

        </div>
        <ul class="list-group list-group-flush">
            <li class="list-group-item">
                @Html.ActionLink("Transferencias nacionales", "Nacionales", "Nacionales", new {
sNumeroTarjeta = ViewBag.sNumeroTarjeta, sSaldo = ViewBag.sSaldo }, new { @class =
"text-decoration-none" })
            </li>
            <li class="list-group-item">
                @Html.ActionLink("Transferencias internacionales", "Menuopcion", "Home", new {
opcion = 2 }, new { @class = "text-decoration-none" })
            </li>
            <li class="list-group-item">
                @Html.ActionLink("Pagos de Servicios", "Menuopcion", "Home", new { opcion = 3 },
new { @class = "text-decoration-none" })
            </li>
            <li class="list-group-item">
                @Html.ActionLink("Verificar saldos", "Menuopcion", "Home", new { opcion = 4 }, new {
@class = "text-decoration-none" })
            </li>
        </ul>
    </div>
</div>
</div>
</div>

```

23. Agregar un nuevo controlador de nombre **NacionalesController**.

24. Dentro de la nueva clase **NacionalesController**, primero referencie a las clases definidas en su carpeta **Models** del namespace de su proyecto, redactando la siguiente sentencia:

```
using CajeroWeb.Models;
```

Luego, reemplace el código predeterminado dentro de esta clase por el contenido siguiente:

```

using CajeroWeb.Models.CajeroWeb.Models;
using Microsoft.AspNetCore.Mvc;

namespace CajeroWeb.Controllers

```

```

{
    public class NacionalesController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        // GET: Nacionales
        public ActionResult Nacionales(string sNumeroTarjeta, double sSaldo)
        {
            ViewBag.sNumeroTarjeta = sNumeroTarjeta;
            ViewBag.sSaldo = sSaldo;
            return View();
        }

        [HttpPost]
        public ActionResult NacionalesNuevo(string sNumeroTarjeta, double? sSaldo, double? cantidad)
        {
            if (string.IsNullOrEmpty(sNumeroTarjeta) || sSaldo == null || cantidad == null)
            {
                return BadRequest("Faltan datos para procesar la transacción.");
            }

            Transaccion nuevaTransaccion = new Transaccion(sNumeroTarjeta, sSaldo.Value);

            if (cantidad.Value > 0)
            {
                nuevaTransaccion.TransferenciaNacional(cantidad.Value);
            }

            double nuevoSaldo = nuevaTransaccion.GetSaldo();

```

```

        return Redirect("~/Menu/Menu?sNumeroTarjeta=" + sNumeroTarjeta + "&sSaldo=" +
nuevoSaldo);
    }

}

}

```

25. Tome como base los pasos 18 al 20 de este procedimiento, para crear una nueva vista para el controlador anterior (**NacionalesController**).

Luego, digite el siguiente código en el archivo correspondiente **Nacionales.cshtml**.

```

@{
    ViewBag.Title = "Nacionales";
    var bancos = new List<SelectListItem>(){
        new SelectListItem { Text = "Banco Agrícola", Value = "0" },
        new SelectListItem { Text = "Banco de América Central", Value = "1" },
        new SelectListItem { Text = "Banco Davivienda", Value = "2", Selected = true },
        new SelectListItem { Text = "Banco Cuscatlán", Value = "3" }
    };
}

<div class="container mt-5">
    <div class="row">
        <!-- Sección izquierda con el formulario -->
        <div class="col-md-8">
            <div class="card shadow">
                <div class="card-body">
                    <h2 class="card-title mb-3">Bienvenido</h2>
                    <p class="card-text">
                        Innovando las transacciones en línea, ponemos a su disposición el cajero ATM online,
                        donde podrá realizar sus pagos y transferencias en línea de una forma rápida y fácil,
                        siempre con la calidad y la confianza que nos caracterizan. Transacciones disponibles
24/7.
                    </p>

```

```

@using (Html.BeginForm("NacionalesNuevo", "Nacionales", FormMethod.Post))
{
    <!-- Mostrar número de tarjeta -->
    <div class="mb-3">
        <label class="form-label">Tarjeta #:</label>
        <p class="form-control-plaintext">@ViewBag.sNumeroTarjeta</p>
        @Html.Hidden("sNumeroTarjeta", (string)ViewBag.sNumeroTarjeta)
    </div>

    <!-- Mostrar saldo -->
    <div class="mb-3">
        <label class="form-label">Saldo:</label>
        <p class="form-control-plaintext">@ViewBag.sSaldo</p>
        @Html.Hidden("sSaldo", (double)ViewBag.sSaldo)
    </div>

    <!-- Selección de banco -->
    <div class="mb-3">
        @Html.Label("misbancos", "Seleccione un banco", new { @class = "form-label" })
        @Html.DropDownList("misbancos", bancos, new { @class = "form-select" })
    </div>

    <!-- Cantidad a transferir -->
    <div class="mb-3">
        @Html.Label("cantidad", "Cantidad:", new { @class = "form-label" })
        @Html.TextBox("cantidad", null, new
        {
            @class = "form-control",
            placeholder = "Ingrese el monto a transferir",
            type = "number",
            step = "0.01",
            min = "0"
        })
    </div>
}

```

```

        })

    </div>

    <!-- Error en caso de fondos insuficientes -->
    @if (ViewBag.Error != null)
    {
        <div class="alert alert-danger">
            @ViewBag.Error
        </div>
    }

    <div class="d-grid">
        <button type="submit" class="btn btn-primary">Transferir</button>
    </div>
}
</div>
</div>
</div>

<!-- Sección derecha del menú -->
<div class="col-md-4">
    <div class="card shadow">
        <div class="card-header bg-primary text-white">
            Opciones del menú
        </div>
        <ul class="list-group list-group-flush">
            <li class="list-group-item">
                @Html.ActionLink("Transferencias nacionales", "someaction", "somecontroller", null,
new { @class = "text-decoration-none" })
            </li>
            <li class="list-group-item">
                @Html.ActionLink("Transferencias internacionales", "someaction", "somecontroller",
null, new { @class = "text-decoration-none" })
            </li>
        </ul>
    </div>
</div>

```

```

        </li>
        <li class="list-group-item">
            @Html.ActionLink("Pagos de Servicios", "someaction", "somecontroller", null, new {
@class = "text-decoration-none" })
        </li>
        <li class="list-group-item">
            @Html.ActionLink("Verificar saldos", "someaction", "somecontroller", null, new {
@class = "text-decoration-none" })
        </li>
    </ul>
</div>
</div>
</div>
</div>

```

26. Para probar los cambios realizados al sitio web, vaya al *Explorador de soluciones* y abra la carpeta **Views ->Home**. Seleccione y abra al archivo de la Vista de inicio (**Index.cshtml**).

Luego, seleccione el navegador web de su preferencia y ejecute el sitio web.

27. Para identificar el usuario y contraseña para acceder a la siguiente Vista del sitio, abra al archivo de la clase **HomeController** y analice su cod. fuente.

Cuando acceda con las credenciales esperadas, la próxima Vista que se cargara en navegador será la de **Menu**, así:

Bienvenido

Innovando las transacciones en línea, ponemos a su disposición el cajero ATM online, donde podrá realizar sus pagos y transferencias en línea de una forma rápida y fácil, siempre con la calidad y la confianza que nos caracterizan. Realice sus transacciones desde la comodidad de su hogar las 24 horas del día, los 7 días de la semana.

Opciones del menú

[Transferencias nacionales](#)

[Transferencias internacionales](#)

[Pagos de Servicios](#)

[Verificar saldos](#)

27. Del menú de enlaces, seleccione la opción *Transferencias nacionales*, para comprobar cómo funcionan las Vistas de esta opción.

The screenshot shows a web application interface for national transfers. On the left is a main form with a blue header 'Bienvenido' and a welcome message. Below this, it displays the card number '123456' and the balance '500'. There is a dropdown menu for selecting a bank, currently showing 'Banco Davivienda'. Below that is a text input field for the transfer amount, with a placeholder 'Ingrese el monto a transferir'. At the bottom of the form is a yellow button labeled 'Transferir'. On the right is a sidebar menu titled 'Opciones del menú' with four items: 'Transferencias nacionales' (highlighted in blue), 'Transferencias internacionales', 'Pagos de Servicios', and 'Verificar saldos'.

28. Realice varias transacciones y confirme que el saldo disponible va disminuyendo según cada monto de operación realizada.

29. Tomando como base nuestra página de “Nacionales”, crear un nuevo Controlador y Vista para las transferencias internacionales. Debe funcionar tal como se muestran en las imágenes a continuación:

The screenshot shows a web application interface for international transfers. The main form on the left has a blue header 'Bienvenido' and a welcome message. It displays the card number '123456' and the balance '500'. There are two dropdown menus: the first for selecting a country, currently showing 'Costa Rica', and the second for selecting a bank, currently showing 'Banco Davivienda'. Below these is a text input field for the transfer amount, with a placeholder 'Ingrese el monto a transferir'. At the bottom of the form is a yellow button labeled 'Transferir'. On the right is a sidebar menu titled 'Opciones del menú' with four items: 'Transferencias nacionales', 'Transferencias internacionales' (highlighted in blue), 'Pagos de Servicios', and 'Verificar saldos'.

28. Realizar una página y un controlador para mostrar el saldo de nuestra cuenta, tal como se muestra en la imagen siguiente (la opción **Verificar saldos**):

The mockup consists of two main parts. On the left, a white box with a blue header 'Bienvenido' contains a paragraph about online transactions and a blue box displaying 'Tarjeta #: 123456' and 'Saldo: 500'. On the right, a blue sidebar menu titled 'Opciones del menú' lists 'Transferencias nacionales', 'Transferencias internacionales', 'Pagos de Servicios', and 'Verificar saldos'.

29. Ahora, proceda a ejecutar la aplicación y realice las respectivas transacciones verificando la integridad de la información.

Ejemplo 2 : Crear una aplicación MVC básica

Objetivo: Crear una aplicación web que muestre una lista de productos estáticos.

1. Configurar el entorno:

- Abre Visual Studio y crea un nuevo proyecto de tipo "ASP.NET Core Web App (Model-View-Controller)".
- El nombre del proyecto para este ejemplo es: Ejemplo01ASP
- Usa .NET 6 o superior.

2. Crear el modelo:

- Crea una clase Product en la carpeta Models.

```
namespace Ejemplo01ASP.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}
```

3. Crear el controlador:

- Crea un controlador ProductsController en la carpeta Controllers.

```
using Ejemplo01ASP.Models;
using Microsoft.AspNetCore.Mvc;

namespace Ejemplo01ASP.Controllers
{
    public class ProductsController : Controller
    {
        public IActionResult Index()
        {
            var products = new List<Product>
            {
                new Product { Id = 1, Name = "Laptop", Price = 999.99m },
                new Product { Id = 2, Name = "Smartphone", Price = 499.99m }
            };
            return View(products);
        }
    }
}
```

4. Crear la vista:

- Crea una carpeta Views/Products y agrega un archivo Index.cshtml.

```
@model IEnumerable<Ejemplo01ASP.Models.Product>
<h2>Lista de Productos</h2>
<table class="table">
    <thead>
        <tr>
            <th>ID</th>
            <th>Nombre</th>
            <th>Precio</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var product in Model)
        {
            <tr>
                <td>@product.Id</td>
                <td>@product.Name</td>
                <td>@product.Price</td>
            </tr>
        }
    </tbody>
</table>
```

5. Ejecutar:

- Ejecuta la aplicación y navega a /Products. Verifica que se muestre la lista de productos.
- Si no tiene certificados válidos, mejor usar HTTP y no HTTPS

Lista de Productos

ID	Nombre	Precio
1	Laptop	999.99
2	Smartphone	499.99

Agregar un formulario para agregar productos

1. Actualizar el controlador:

- Agregamos métodos para mostrar y procesar un formulario en ProductsController.

```
using Ejemplo01ASP.Models;
using Microsoft.AspNetCore.Mvc;

namespace Ejemplo01ASP.Controllers
{
    public class ProductsController : Controller
    {
        private static List<Product> products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Price = 999.99m },
            new Product { Id = 2, Name = "Smartphone", Price = 499.99m },
            new Product { Id = 3, Name = "SmartWatch", Price = 65.99m },
            new Product { Id = 4, Name = "Tablet", Price = 325.54m }
        };

        public IActionResult Index()
        {
            return View(products);
        }

        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Product product)
        {
            if (ModelState.IsValid)
            {
                product.Id = products.Count + 1;
                products.Add(product);
                return RedirectToAction("Index");
            }
            return View(product);
        }
    }
}
```

```
}  
}
```

2. Crear la vista del formulario:

- Crea un archivo Create.cshtml en Views/Products.

```
@model Ejemplo01ASP.Models.Product  
<h2>Agregar Producto</h2>  
<form asp-action="Create" method="post">  
    <div>  
        <label asp-for="Name">Nombre:</label>  
        <input asp-for="Name" class="form-control" />  
    </div>  
    <div>  
        <label asp-for="Price">Precio:</label>  
        <input asp-for="Price" class="form-control" />  
    </div>  
    <button type="submit" class="btn btn-primary">Agregar</button>  
</form>
```

3. Actualizar la vista Index:

- Agrega un enlace para crear un nuevo producto en Index.cshtml.

```
using Ejemplo01ASP.Models;  
using Microsoft.AspNetCore.Mvc;  
  
namespace Ejemplo01ASP.Controllers  
{  
    public class ProductsController : Controller  
    {  
        private static List<Product> products = new List<Product>  
        {  
            new Product { Id = 1, Name = "Laptop", Price = 999.99m },  
            new Product { Id = 2, Name = "Smartphone", Price = 499.99m },  
            new Product { Id = 3, Name = "SmartWatch", Price = 65.99m },  
            new Product { Id = 4, Name = "Tablet", Price = 325.54m }  
        };  
  
        public IActionResult Index()  
        {  
            return View(products);  
        }  
  
        public IActionResult Create()  
        {  
            return View();  
        }  
  
        [HttpPost]  
        public IActionResult Create(Product product)  
        {  
            if (ModelState.IsValid)  
            {  
                product.Id = products.Count + 1;  
            }  
        }  
    }  
}
```

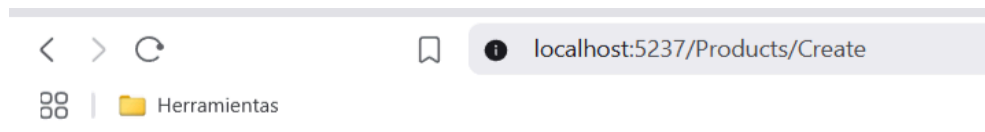
```

        products.Add(product);
        return RedirectToAction("Index");
    }
    return View(product);
}
}
}

```

Probar:

- Ejecuta la aplicación, navega a /Products/Create, agrega un producto y verifica que aparezca en la lista.

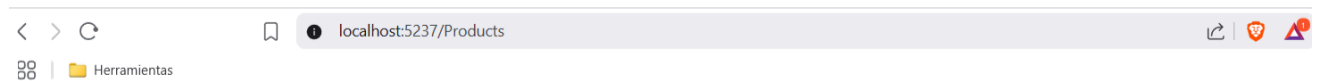


Ejemplo01ASP Home Privacy

Agregar Producto

Nombre:

Precio:



Ejemplo01ASP Home Privacy

Lista de Productos

Lista de Productos

ID	Nombre	Precio
1	Laptop	999.99
2	Smartphone	499.99
3	SmartWatch	65.99
4	Tablet	325.54
5	Mouse inalambrico	15.35

Ahora vamos a agregar validaciones y estilos

1. Agregar validaciones al modelo:

- Modifica la clase Product para incluir validaciones.

```
using System.ComponentModel.DataAnnotations;

namespace Ejemplo01ASP.Models
{
    public class Product
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "El nombre es obligatorio")]
        [StringLength(100, ErrorMessage = "El nombre no puede exceder 100 caracteres")]
        public string Name { get; set; }

        [Required(ErrorMessage = "El precio es obligatorio")]
        [Range(0.01, 10000, ErrorMessage = "El precio debe estar entre 0.01 y 10000")]
        public decimal Price { get; set; }
    }
}
```

2. Actualizar la vista Create:

- Agrega validaciones en Create.cshtml.

```
@model Ejemplo01ASP.Models.Product
<h2>Agregar Producto</h2>
<form asp-action="Create" method="post">
    <div class="form-group">
        <label asp-for="Name" class="control-label"></label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-danger"></span>
    </div>
    <div class="form-group">
        <label asp-for="Price" class="control-label"></label>
        <input asp-for="Price" class="form-control" />
        <span asp-validation-for="Price" class="text-danger"></span>
    </div>
    <button type="submit" class="btn btn-primary">Agregar</button>
</form>
@section Scripts {
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.6.0/jquery.min.js"></script>
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/jquery-validate/1.19.3/jquery.validate.min.js"></script>
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/jquery-validation-unobtrusive/3.2.12/jquery.validate.unobtrusive.min.js"></script>
}
```

Probar:

- Verifica que las validaciones funcionen (ej., no aceptar nombres vacíos o precios inválidos).

V. Ejercicios complementarios

El instructor evaluará el desempeño de su trabajo dentro la clase. Estos ejercicios se evaluarán en la misma hora de clase.

Para el ejemplo 1:

1. Agregar una acción Details en ProductsController que muestre los detalles de un producto seleccionado por su ID. Crear una vista Details.cshtml para mostrar la información.
2. Implementar una acción Edit para modificar los datos de un producto existente. Asegurase de incluir validaciones en el formulario de edición.
3. Agregar una acción Delete que permita eliminar un producto de la lista. Incluir una confirmación antes de eliminar.

VI. Análisis de resultados

Para la próxima semana

- Crea una aplicación MVC para gestionar una lista de tareas (To-Do List). Incluye funcionalidades para crear, listar, editar y eliminar tareas. Usa validaciones para asegurar que las tareas tengan un título y una fecha de vencimiento.
- Desarrolla una aplicación MVC que simule un sistema de reservas de libros en una biblioteca. Incluye modelos para libros y usuarios, y permite listar, reservar y cancelar reservas.
- Investiga cómo integrar una base de datos (como SQL Server o SQLite) con ASP.NET MVC usando Entity Framework Core. Implementa una versión de la aplicación de productos que almacene los datos en una base de datos en lugar de una lista en memoria.
- Investiga qué es y cómo se implementa el **Routing** en ASP.NET MVC. Escribe un ejemplo de cómo se define una ruta personalizada.

V. Bibliografía

1. Microsoft Learn - Documentación oficial de .NET, <https://learn.microsoft.com/es-es/dotnet/> Incluye tutoriales de C#, WinForms, y controles UI.
2. Troelsen, A. (2023). Pro C# 10 with .NET 6. Apress. Libro de referencia para sintaxis avanzada, patrones de diseño y buenas prácticas.
3. Richter, J. (2023). CLR via C#. Microsoft Press. Fundamentos del runtime de .NET y gestión de memoria.
4. Stack Overflow - Comunidad .NET, <https://stackoverflow.com/questions/tagged/c%23>. Resolución de errores comunes y mejores prácticas validadas por la comunidad.
5. DevExpress - WinForms UI Best Practices, <https://www.devexpress.com/winforms/> Guías de diseño de interfaces y uso de controles avanzados.

Recursos adicionales

Extensión de Visual Studio:

[Microsoft XAML Controls Gallery](#) para inspirar diseños modernos.

Video-tutoriales:

Playlist de YouTube sobre WinForms (ej: "Canal de Programming with Mosh").

Libros:

- Troelsen, A. (2022). Pro C# 10 with .NET 6. Apress.
- Richter, J. (2020). CLR via C#. Microsoft Press.